BABA IS YOU

LAHOUD Steven

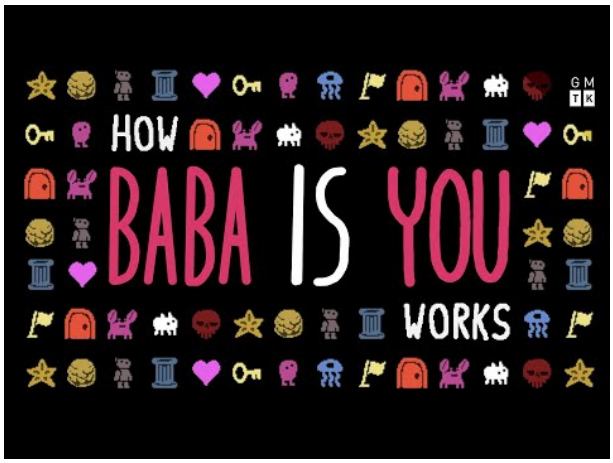
BOUZOUMITA Yasmine

Université
Paris Cité

Sommaire

1. Introduction 3
2. Architecture Logicielle 4
    2.1. Diagramme de classes (UML) 4
    2.2. Application de design pattern 4
3. Implémentation Technique C++ 4
    3.1. Gestion manuelle de la mémoire (sans auto) 4
    3.2. Gestion de la copie 5
    3.3. Surcharche d'opérateurs ([]) 5
4. Logique du Jeu 5
    4.1. Analyse des règles 5
    4.2. Algorithme de déplacement et interactions 6
5. Extensions 6
    5.1. Couleurs pour les niveaux 6
    5.2. PULL/TELE 6
6. Bilan technique 6
    6.1. Problèmes connus et limites 6
    6.2. Améliorations possibles 7

1. Introduction

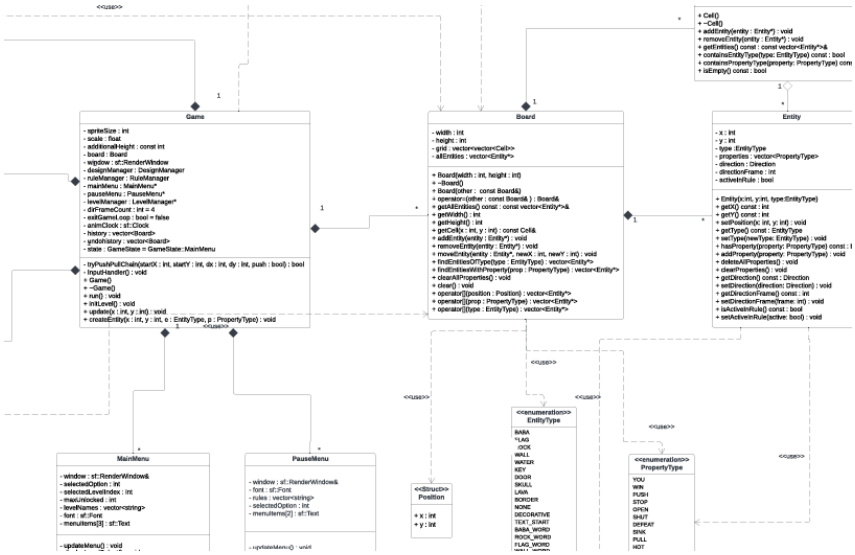


Ce projet consiste en l'implémentation d'une version du jeu **Baba Is You** en C++. L'objectif est de reproduire les mécanismes de manipulation des règles au sein d'une grille 2D. Pour cela nous avons créé un dépôt GitHub afin de sauvegarder notre travail au fur et à mesure de son avancement.
Tout au long du projet, nous avons suivi des étapes particuliers:

- Maîtriser le **système du jeu** ainsi que le fonctionnement de toutes les règles envisageables (PUSH/STOP…).
- Construire une **architecture logicielle** (UML) adaptée et qui répond le plus possible aux attentes du sujet.
- Choisir les **méthodes techniques** à utiliser et les règles clés à suivre (ex : ce qui est créé doit être détruit, ne copiez que ce qui est nécessaire…)
- Se familiariser avec l'**interface graphique SFML** ainsi que son fonctionnement sur « Windows » avec les bonnes versions.
- Ajouter de diverses extensions comme par exemple les règles incluant TELE et HOT/MELT.

2. Architecture Logicielle

2.1. Diagramme de classes (UML)



L'architecture finale de notre projet, illustrée par le diagramme de classe ci-dessus, est le résultat d'un processus itératif.
Optant au début de faire une classe mère « **Entity** » avec toutes les entités (ex : BABA, WALL..) comme classes filles, cette idée à rapidement basculé vers la création de deux enum, **EntityType** et **PropertyType** ; cette distinction est importante, en particulier pour l'implémentation de règles du type « WALL IS BABA » qui est alors faite de manière dynamique sans devoir détruire et recréer l'objet en mémoire.
La représentation du monde est gérée par la classe **Board** qui contient une grille composée de **Cell** et un vecteur de toutes ses **Entity**.
La classe **Game** tendait à devenir une God Class, c'est pour cela qu'on a pris l'initiative de la diviser en classes ayant des rôles clairs : **RuleManager** devient responsable de l'analyse des phrases et de l'application des propriétés ; quant à elle, **DesignManager** prend en charge toute la partie graphique (SFML) du projet. LevelManager, MainMenu et PauseMenu prennent eux aussi une partie isolé du projet qui est à la fin utilisée par le chef d'orchestre **Game**.
Cette structure nous a permis de travailler sur les règles du jeu (RuleManager) sans impacter le moteur de rendu (DesignManager), facilitant ainsi le travail en binôme et le débogage.

2.2. Application de design pattern

Pour garantir l'unicité du moteur de jeu, nous avons appliqué le patron de conception « **Singleton** » à la classe **Game**. Ce choix assure qu'une seule instance du jeu est créée.
Nous avons également respecté la séparation des responsabilités proche du modèle **MVC** (Modèle-Vue-Contrôleur) en isolant les données (Board), l'affichage (DesignManager) et la logique (RuleManager).

3. Implémentation Technique C++

3.1. Gestion manuelle de la mémoire (sans auto)

Chaque allocation dynamique via l'opérateur **new** est couplée à une libération via **delete** dans le destructeur de la classe possédant la ressource.

En titre d'exemple, la classe **Game** est propriétaire des managers (**LevelManager**, **DesignManager**) et donc c'est elle qui les détruit à la fermeture. Pareil pour la classe **Board** qui est responsable du cycle de vie des **entités** qu'elle contient.

3.2. Gestion de la copie

Cette partie a joué un rôle crucial dans l'implémentation de **undo/redo**. En effet, puisque notre classe Board gère des entités via des pointeurs, on a redéfini son **constructeur de copie** qui alloue une nouvelle mémoire et clone chaque entité une par une.
Cette approche garantit que chaque sauvegarde dans l'historique est totalement indépendante de l'état actuel du jeu ce qui permet de réaliser les actions de **undo** et **redo**.

3.3. Surcharche d'opérateurs ([])

Comme mentionné dans le sujet, on a surchargé l'opérateur [] dans la classe Board. Ceci à facilité par la suite le fait de récupérer des entités se trouvant dans une position précise, ayant un type ou une propriété définis. En effet, cette surcharge a simplifié l'écriture des algorithmes de règles et de déplacement ce qui rend la lecture du code plus fluide.

4. Logique du Jeu

4.1. Analyse des règles

À chaque niveau du jeu Baba Is You, l'objectif est de faire en sorte qu'une entité YOU atteigne une entité WIN. C'est la seule façon de gagner le niveau. Si la règle concernant YOU est détruite, on ne peut plus rien faire, sauf réinitialiser le niveau.

Dire qu'une entité *meurt* signifie simplement qu'elle disparaît du niveau en raison des règles qui la concernent.

Dans le jeu, il existe deux types de mécanismes pour les règles de base :
- **Entité IS Propriété** : chaque entité du type spécifié possède la propriété mentionnée et se comporte en conséquence, tant que la règle est active.
- **Entité IS Entité** : dès que cette règle est activée, toutes les entités du genre à gauche se transforment en entités du genre à droite, en héritant de leurs propriétés actives éventuelles ; bien sûr, toutes les règles concernant la première entité deviennent inutiles puisque toutes les instances de cette entité ont disparu. Comme il s'agit d'une règle à effet instantané, même si elle est détruite par la suite, cela ne rétablit pas les entités précédentes.

Les entités sont simplement les différents personnages, objets et blocs qui s'affichent à l'écran, avec lesquels l'on peut ou non interagir.

En ce qui concerne les propriétés, si une entité possède la propriété :
- YOU : le joueur contrôle ses déplacements à l'aide des flèches du clavier.
- WIN : elle permet au joueur de gagner le niveau s'il se déplace jusqu'à sa position.
- PUSH : elle devient tangible, le joueur peut la déplacer en la poussant (il doit se placer derrière elle dans la position où il veut aller).
- STOP : elle ne peut pas être déplacée, on ne peut rien superposer dessus ; c'est une véritable barrière.
- PULL : elle devient tangible, le joueur peut la déplacer en la tirant, il se place devant elle et va dans la direction opposée, et elle le suit.
- DEFEAT : elle provoque la mort du joueur s'il se trouve à sa position.
- SINK : si le joueur ou un objet tangible se trouve à sa position, il meurt et elle meurt avec lui.
- SHUT : si une entité OPEN se trouve à sa position, elle meurt.
- OPEN : elle peut tuer une entité SHUT, en mourant avec elle.

- TELE : le joueur ou les entités à sa position apparaissent à la position de son jumeau à chaque action.
- HOT : elle peut faire mourir les entités MELT.
- MELT : si elle se trouve à la même position qu'une entité HOT, elle meurt.

4.2. Algorithme de déplacement et interactions

Au cœur de la logique du jeu, on trouve l'algorithme qui permet de gérer comment les entités se déplacent sur le plateau et comment elles interagissent entre elles selon les règles définies.
Lorsque la méthode `Game::InputHandler()` détecte que le joueur souhaite faire un déplacement, la méthode `Game::update(int x, int y)` est appelée ; elle va calculer la position cible sur le plateau et s'assurer qu'elle rentre dans les bords du plateau. Si la position cible contient une autre entité, la méthode détermine l'interaction à appliquer, que cela soit de pousser un objet ou de bloquer le mouvement selon les règles du plateau.
Après que le déplacement a été effectué, le **RuleManager** va s'occuper d'appliquer toutes les règles présentes sur le plateau et de gérer les interactions spéciales (SINK, MELT, OPEN…). Le **DesignManager** va garantir que l'état logique du plateau et l'affichage restent synchronisés.
Toutes ces étapes se répètent dans `Game::run()`, la boucle principale du jeu.

5. Extensions

5.1. Couleurs pour les niveaux

Afin de donner un retour visuel sur la progression du joueur de manière immédiate, nous avons enrichi le menu principal d'un code couleur:
- **Vert** : niveau validé avec succès.
- **Blanc** : niveau courant disponible.
- **Gris** : niveaux futurs, pas encore accessibles (pour aller au niveau j on dois avoir fais tous les niveaux d'avant).

5.2. PULL/TELE

Nous avons ajouté des propriétés intéressantes comme **TELE** et **PULL** : l'implémentation de ces deux propriétés n'a pas été simple contrairement à ce qu'on pensait intuitivement. Pour **TELE** on a pensé à une solution afin de régler le problème de boucle infinie, un effet ping-pong qui se déroule sans fin (sans parler de la possibilité de blocage et que l'objet ne puisse pas alors se téléporter).
Quant à **PULL**, complémentaire à PUSH, c'est une propriété qui permet au joueur de tirer des objets derrière lui. Lors d'un déplacement, l'algorithme vérifie récursivement la chaîne d'entités situées à l'opposé du mouvement pour déterminer si elles doivent suivre la trajectoire du joueur.

6. Bilan technique

6.1. Problèmes connus et limites

En raison de contraintes de temps lors de la phase finale de développement, on a pas pu optimiser à 100% la structure complète du projet et la présence de **méthodes longues** en témoigne. En effet, bien que fonctionnel, le code nécessiterait une phase de **refactoring** pour découper ces fonctions pour que ça soit plus compréhensible et respecter davantage les principes de responsabilité unique.
D'autre part, la fenêtre principale de l'application possède des dimensions fixes et ne peut actuellement être redimensionnée par l'utilisateur. Cette contrainte a été maintenue pour garantir la stabilité de l'affichage des composants graphiques dans cette version.

6.2. Améliorations possibles

En partant de notre code, il ne serait pas très compliqué d'implémenter un système de sauvegarde de sorte que même en fermant le jeu on ne perde pas le progrès fait (en créant bien-sûr une option pour effacer ce fichier de sauvegarde si on veut recommencer le jeu).
Nous savons aussi que d'autres patrons de conception pouvaient être implémentés (comme State pour les menus) mais la contrainte de temps nous à empêché de l'implémenter.
On n'a pas non plus eu le temps de s'occuper des règles de forme AND, mais ces dernières ainsi que la règle HAS (lors de la destruction d'une entité, une autre apparaît à sa place) ne seraient pas difficiles à mettre en place grâce à la structure de notre jeu.

Références:





